

Multilayer Perceptrons, Part 2

Advanced Machine Learning

Sam Ogden

DRAFT

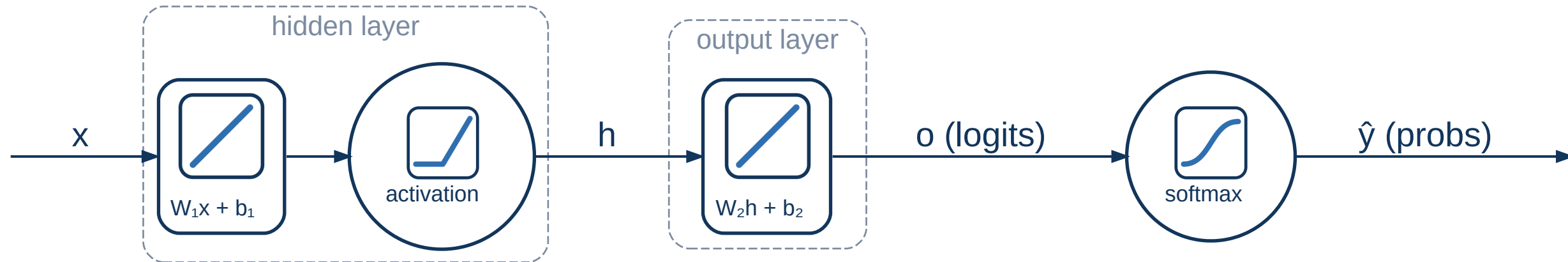
This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- See where each local derivative lives along the model, then chain them
- Follow the error backward one neuron at a time, into a hidden layer
- Walk the full backward pass and one SGD update, on the example from last time

Recap: the forward pass

Where we left off



Same fruit classifier, same example: $x = [1.2, 0.2]$, true class banana.

stage	value
$h_{\text{pre}} = W_1x + b_1$	$[1.4, -0.8]$
$h = \text{ReLU}(h_{\text{pre}})$	$[1.4, 0]$
$o = W_2h + b_2$	$[0.7, -0.42, 0.28]$
$\hat{y} = \text{softmax}(o)$	$[0.5042, 0.1645, 0.3313]$
$L = -\log \hat{y}_{\text{banana}}$	1.8048

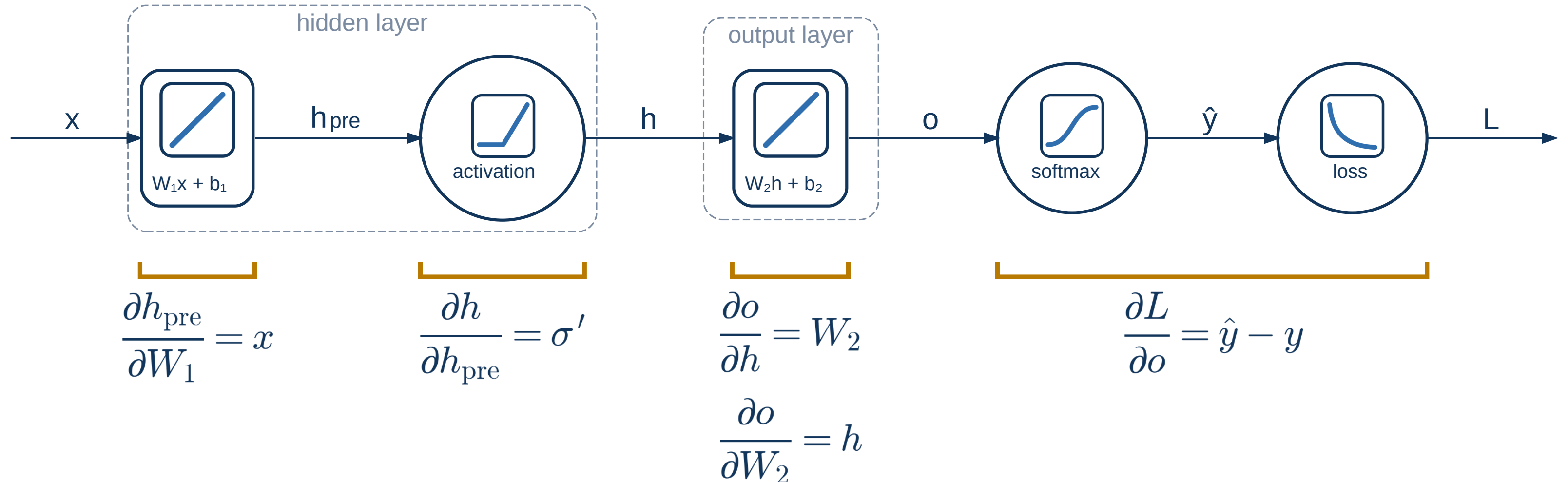
The Big Question

The loss only touches W_1 and b_1
indirectly — how do we figure out
how wrong they were?

Gradients: the rough picture

Which derivative lives where

Local derivatives along the model



- Each bracket marks one stretch of the model and the one small derivative for it — computed from only what's inside that stretch

Chain them together

$$\frac{\partial L}{\partial W_2} = \underbrace{\frac{\partial L}{\partial o}}_{\hat{y}-y} \underbrace{\frac{\partial o}{\partial W_2}}_h$$
$$\frac{\partial L}{\partial W_1} = \underbrace{\frac{\partial L}{\partial o}}_{\hat{y}-y} \underbrace{\frac{\partial o}{\partial h}}_{W_2} \underbrace{\frac{\partial h}{\partial h_{\text{pre}}}}_{\sigma'} \underbrace{\frac{\partial h_{\text{pre}}}{\partial W_1}}_x$$

- Multiply the local derivatives along the path from the loss back to the weight
- W_2 is one link from the loss; W_1 is three links away — same rule, more pieces
- Read these as a map: the next slides work out exactly how the pieces multiply, one neuron at a time

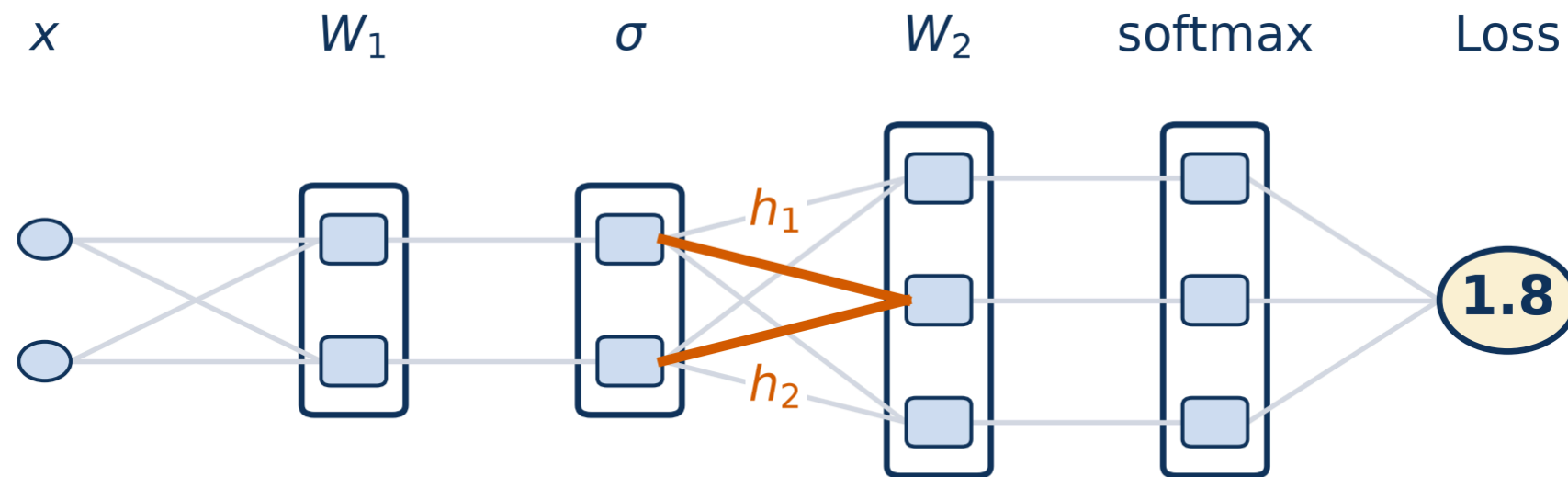
The Big Idea

The chain rule is the key to this – we just follow each step backwards through the network

Gradients: the in-depth picture

One neuron, one weight at a time

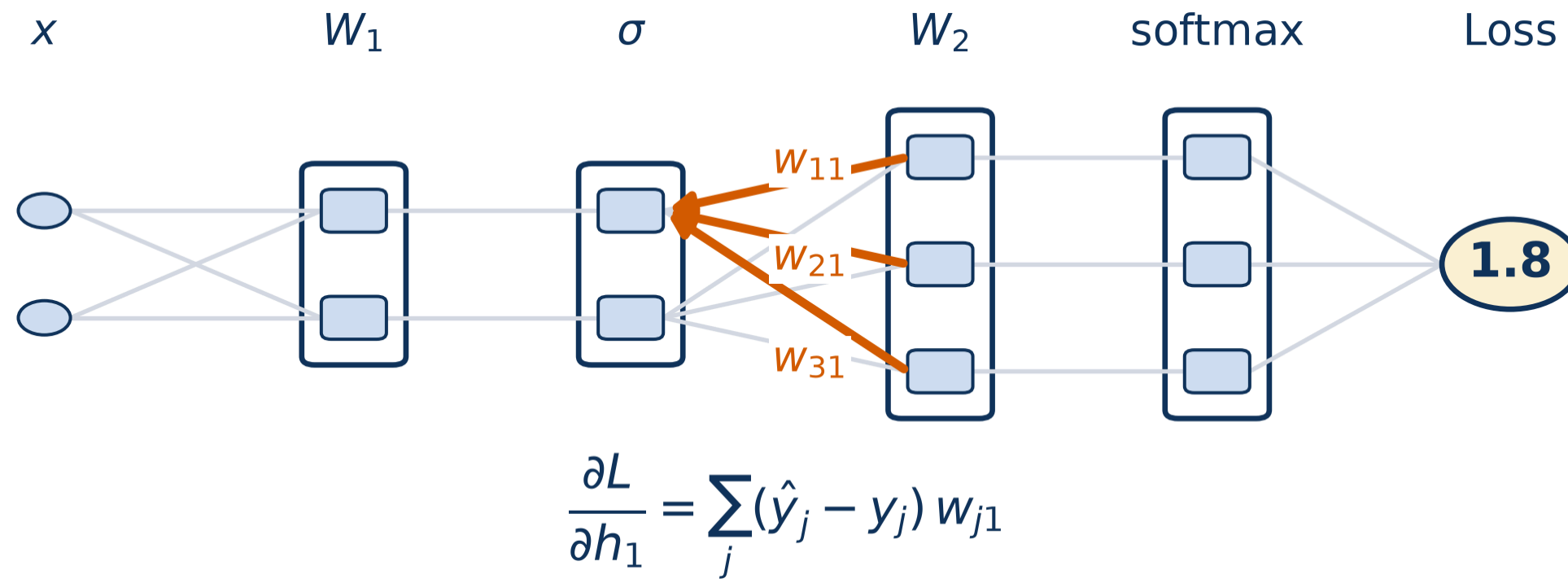
The output layer's weights



$\frac{\partial o_j}{\partial w_{jk}} = h_k$ for the wire from h_k into output neuron j (here, the middle one)

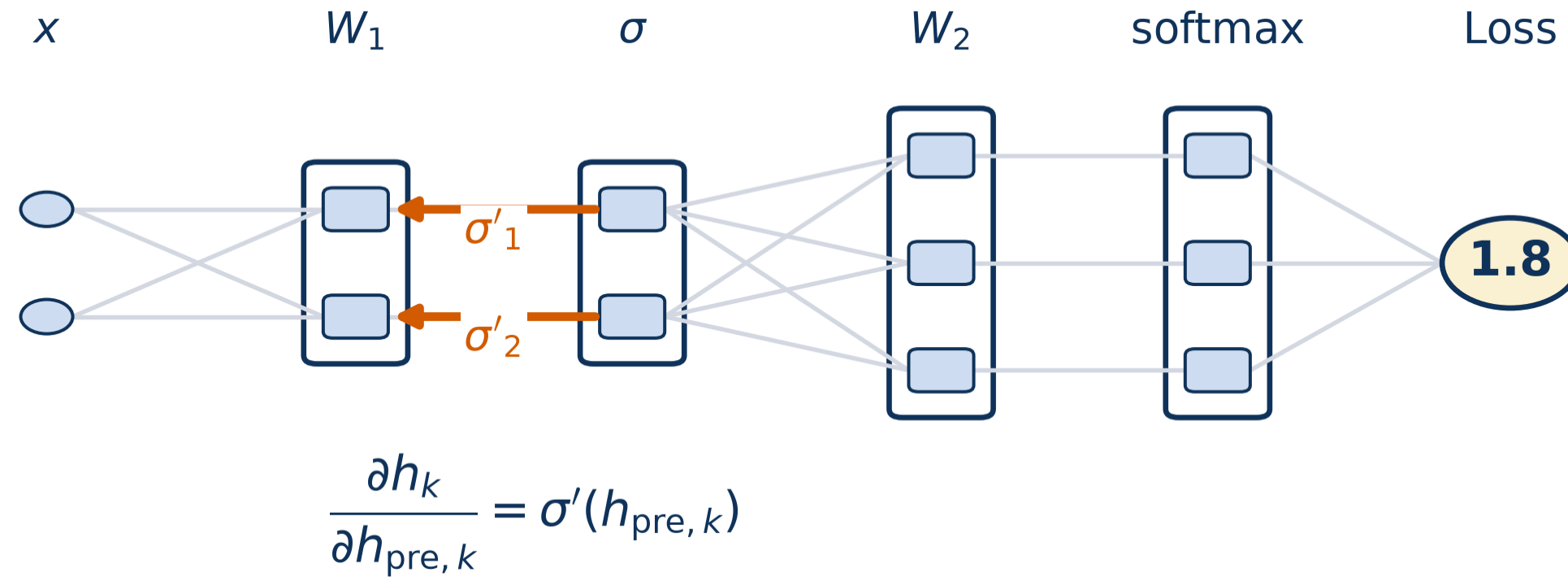
- Each weight w_{jk} only affects its own output o_j , scaled by the value it multiplies, so $\frac{\partial o_j}{\partial w_{jk}} = h_k$
- Chain it on: $\frac{\partial L}{\partial w_{jk}} = (\hat{y}_j - y_j) h_k$, last lecture's formula with h in place of x

Passing the error to a hidden neuron



- The hidden value h_1 feeds every output neuron, so it collects blame from all of them, each weighted by w_{j1}
- In matrix form, that sum is $(W_2)^\top (\hat{y} - y)$

Through the activation



- Each hidden value's error is scaled by its own neuron's activation slope, independently of the others
- Written for the whole layer: $\frac{\partial L}{\partial h_{\text{pre}}} = \frac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}})$ ($\odot$: elementwise)

ReLU's slope

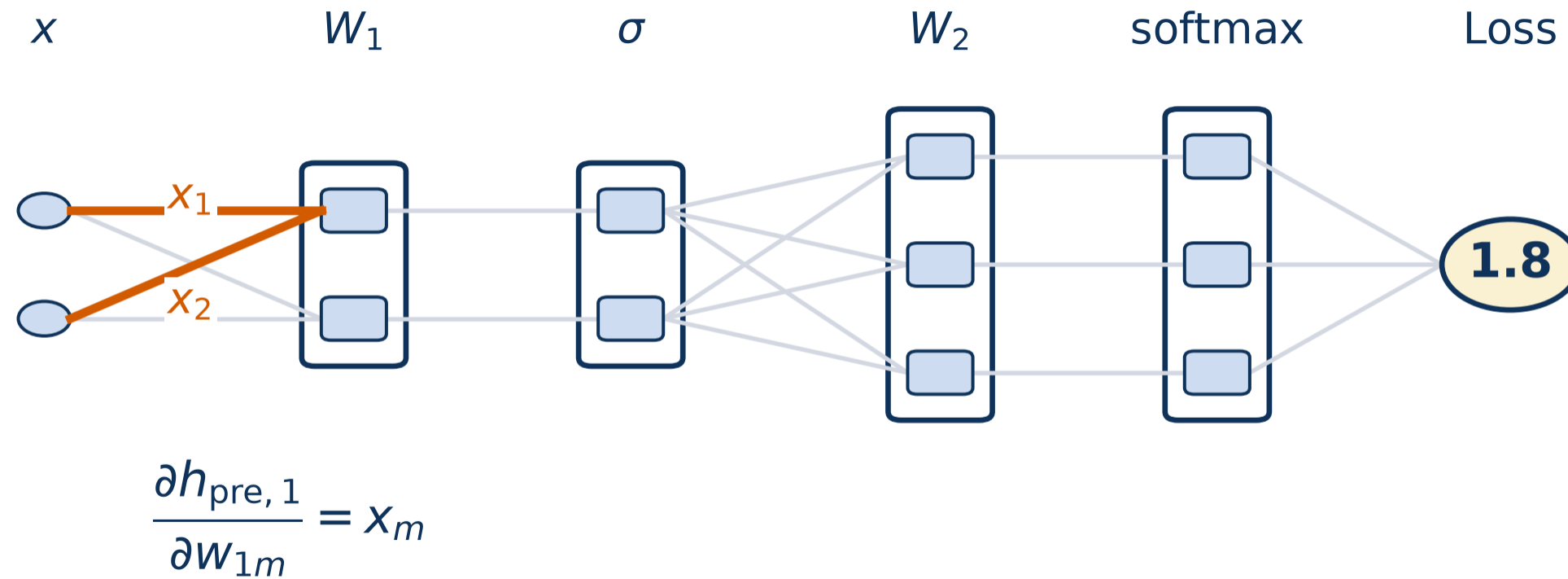
- For ReLU, the slope is especially simple:

$$\sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

! Important

If a neuron's pre-activation was ≤ 0 , ReLU zeroed it going forward — and now its slope is 0 too, so **no gradient passes through it at all**, no matter how large the incoming error was

The hidden layer's weights



- Same move as the output layer, with x feeding the neuron: $\frac{\partial h_{\text{pre},1}}{\partial w_{1m}} = x_m$
- Chain it on: $\frac{\partial L}{\partial w_{1m}} = \frac{\partial L}{\partial h_{\text{pre},1}} \cdot x_m$

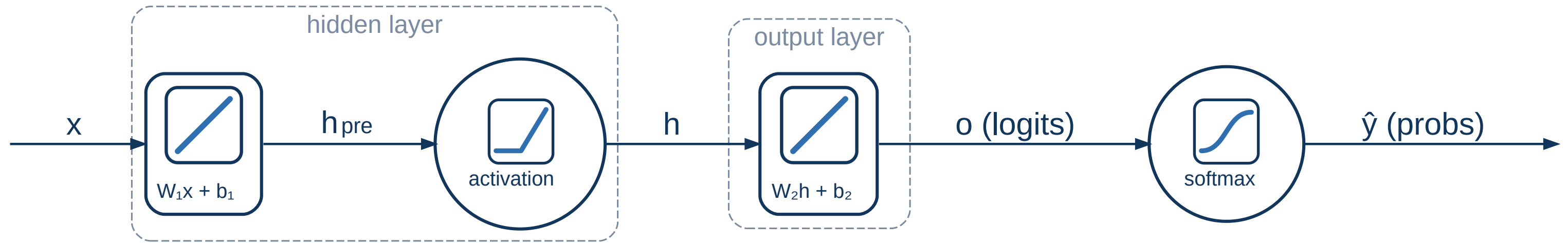
The same rule, in matrix form

quantity	matrix form
$\frac{\partial L}{\partial o}$	$\hat{y} - y$
$\nabla W_2, \nabla b_2$	$(\hat{y} - y) h^\top, \hat{y} - y$
$\frac{\partial L}{\partial h}$	$(W_2)^\top (\hat{y} - y)$
$\frac{\partial L}{\partial h_{\text{pre}}}$	$\frac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}})$
$\nabla W_1, \nabla b_1$	$\frac{\partial L}{\partial h_{\text{pre}}} x^\top, \frac{\partial L}{\partial h_{\text{pre}}}$

Every line is “the error flowing into a layer, times what fed into it” — the same local rule, once per layer, moving backward

Worked example: the backward pass

Setup, recalled from last lecture



x

h_{pre}

h

o

$\hat{y}$

$[1.2, 0.2]$

$[1.4, -0.8]$

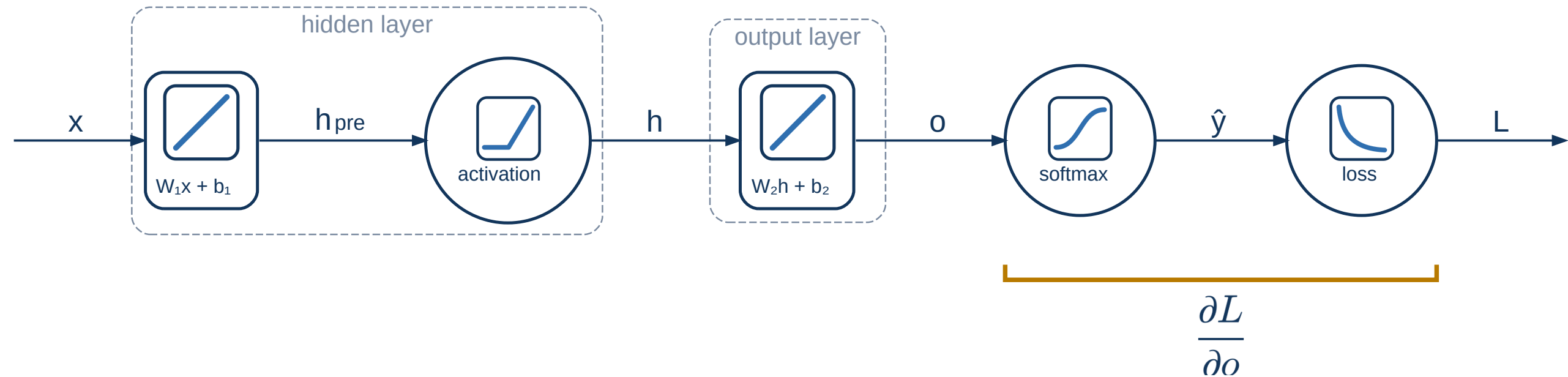
$[1.4, 0]$

$[0.7, -0.42, 0.28]$

$[0.50, 0.16, 0.33]$

True class: banana, so $y = [0, 1, 0]$ and $L \approx 1.8048$

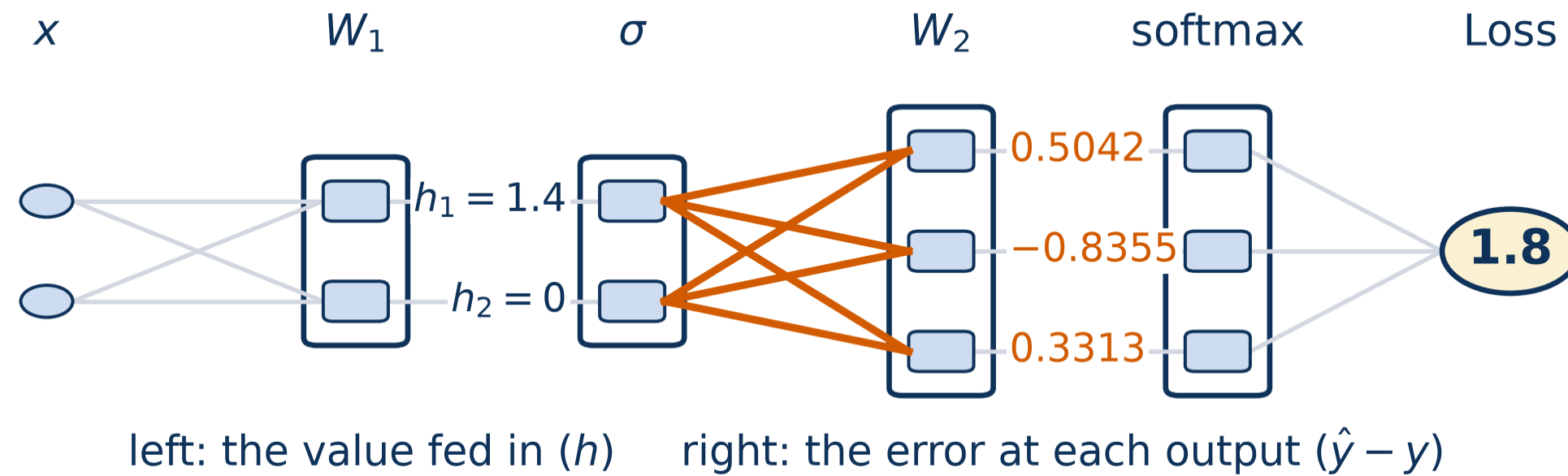
Step 1: start at the loss



$$\begin{aligned}\frac{\partial L}{\partial o} &= \hat{y} - y \\ &= [0.5042, 0.1645, 0.3313] - [0, 1, 0] \\ &= [0.5042, -0.8355, 0.3313]\end{aligned}$$

- We subtract the whole vector y : only banana's entry loses a 1 (the others subtract 0). Negative means push that logit up, positive means push it down

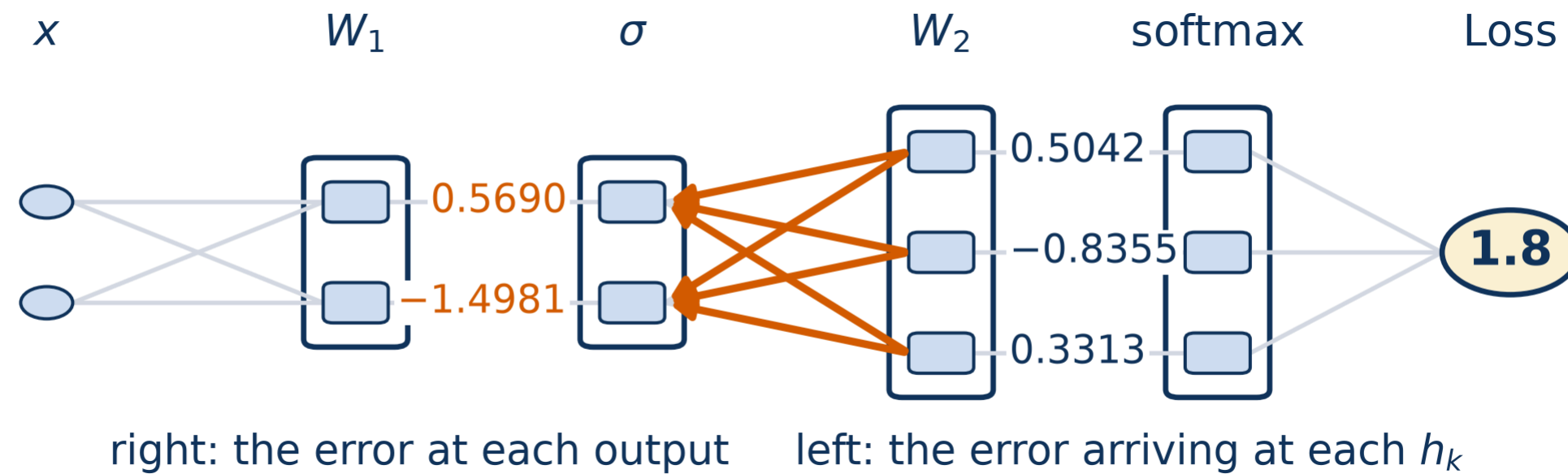
Step 2: how should the output weights change?



$$\nabla W_2 = \begin{bmatrix} 0.5042 \\ -0.8355 \\ 0.3313 \end{bmatrix} \begin{bmatrix} 1.4 & 0 \end{bmatrix} = \begin{bmatrix} 0.7059 & 0 \\ -1.1697 & 0 \\ 0.4638 & 0 \end{bmatrix}$$

- Each weight's gradient is the error at its output neuron times the value that fed it. Column 2 is 0 because $h_2 = 0$; ∇b_2 is just the error vector

Step 3: passing the error back to h

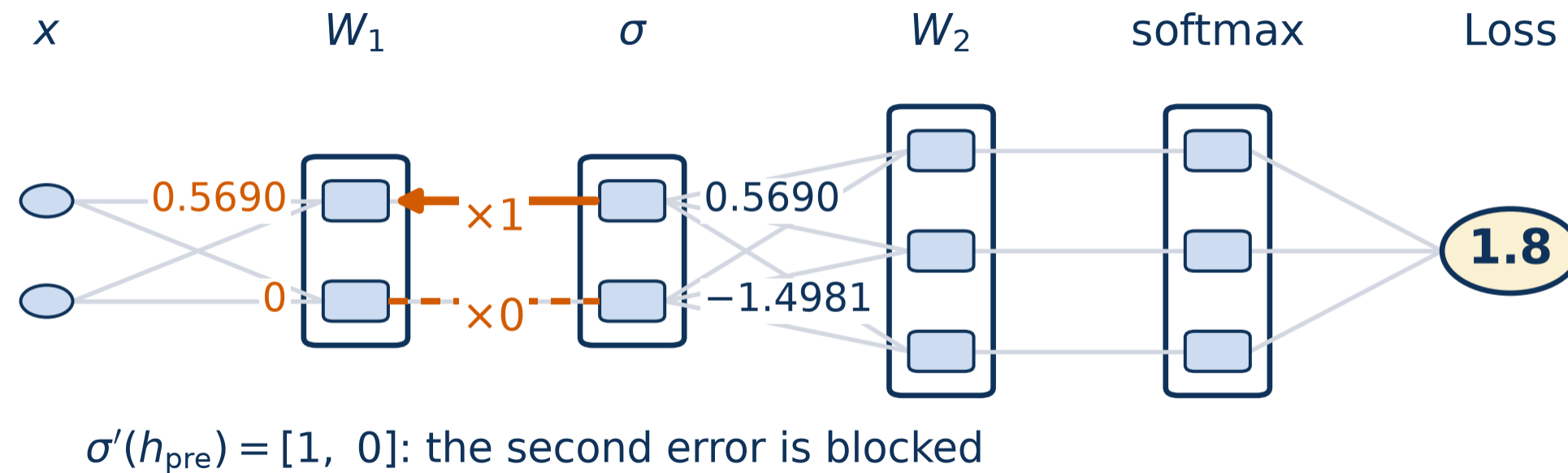


$$\frac{\partial L}{\partial h_1} = (0.5)(0.5042) + (-0.3)(-0.8355) + (0.2)(0.3313) = 0.5690$$

$$\frac{\partial L}{\partial h_2} = (1)(0.5042) + (2)(-0.8355) + (-1)(0.3313) = -1.4981$$

- Each output neuron sends its error back along its weights, and h_k adds up what arrives

Step 4: back through ReLU

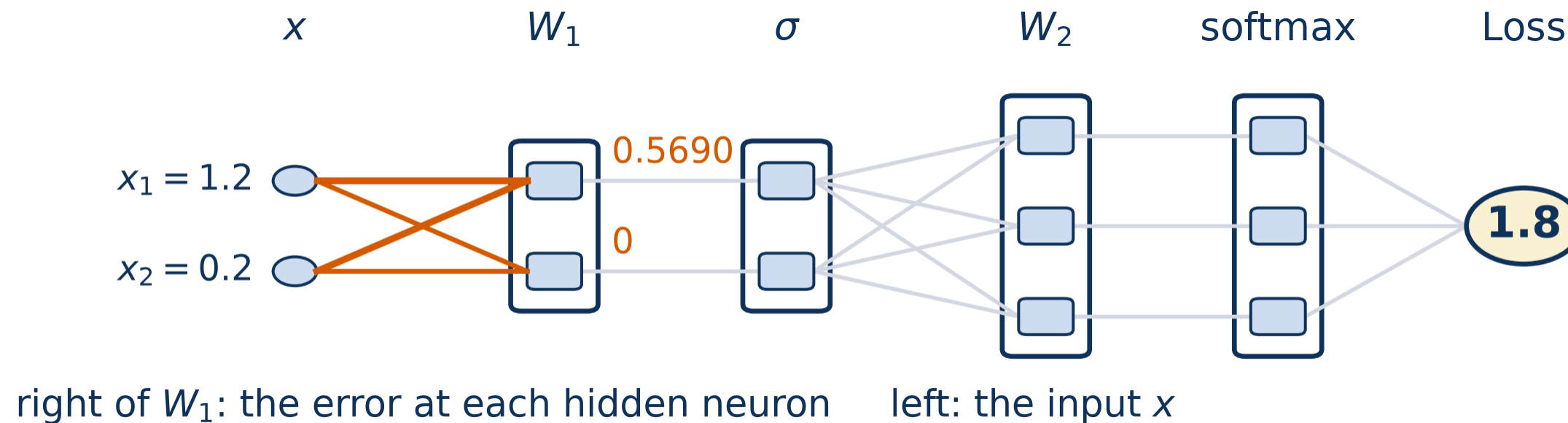


$$\frac{\partial L}{\partial h_{\text{pre},1}} = 0.5690 \times \sigma'(1.4) = 0.5690 \times 1 = 0.5690$$

$$\frac{\partial L}{\partial h_{\text{pre},2}} = -1.4981 \times \sigma'(-0.8) = -1.4981 \times 0 = 0$$

- ReLU's slope is 1 for positive inputs, 0 for negative: h_2 was clamped to 0, so its error is discarded

Step 5: how should the hidden weights change?



$$\nabla W_1 = \begin{bmatrix} 0.5690 \\ 0 \end{bmatrix} \begin{bmatrix} 1.2 & 0.2 \end{bmatrix} = \begin{bmatrix} 0.6828 & 0.1138 \\ 0 & 0 \end{bmatrix}$$

- Same move as Step 2: error at each hidden neuron times the input that fed it. Row 2 is 0 because its error was discarded at ReLU; $\nabla b_1 = [0.5690, 0]$

The update, and what backprop actually is

Updating the output layer

Same rule as always, $\theta \leftarrow \theta - \eta \nabla \theta$, with $\eta = 0.5$:

$$W_2 - 0.5 \nabla W_2 = \begin{bmatrix} 0.5 & 1 \\ -0.3 & 2 \\ 0.2 & -1 \end{bmatrix} - 0.5 \begin{bmatrix} 0.7059 & 0 \\ -1.1697 & 0 \\ 0.4638 & 0 \end{bmatrix} = \begin{bmatrix} 0.1471 & 1 \\ 0.2848 & 2 \\ -0.0319 & -1 \end{bmatrix}$$

$$b_2 - 0.5 \nabla b_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5042 \\ -0.8355 \\ 0.3313 \end{bmatrix} = \begin{bmatrix} -0.2521 \\ 0.4177 \\ -0.1656 \end{bmatrix}$$

Updating the hidden layer

$$W_1 - 0.5 \nabla W_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} - 0.5 \begin{bmatrix} 0.6828 & 0.1138 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.6586 & 0.9431 \\ -1 & 2 \end{bmatrix}$$

$$b_1 - 0.5 \nabla b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5690 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.2845 \\ 0 \end{bmatrix}$$

- The second row of W_1 is unchanged: a zero gradient means zero update, whatever η is

Did the update help?

Run the same forward pass again, with the updated parameters:

$$\mathbf{o}_{\text{new}} = [-0.15, 0.6155, -0.1878], \quad \hat{\mathbf{y}}_{\text{new}} = [0.2431, 0.5228, 0.2341]$$

$$L_{\text{new}} = -\log(0.5228) \approx 0.6486$$

- Loss went from **1.8048** $\rightarrow$ **0.6486** — a bigger drop than the single-layer classifier managed
- And banana now **wins**: 0.5228 is the highest probability, up from third place. One step, two layers, already flipped the decision

What backpropagation actually is

Backpropagation is the chain rule, applied once per layer, moving backward from the loss.

- Nothing here was a special “neural network trick” — every step was ordinary chain-rule calculus, applied to one link in the chain at a time
- This generalizes: a network with 10 hidden layers uses the exact same idea, just with more links — compute the error at the output, then repeatedly ask “what was this layer’s input, and how did it get to me?”

Summary

The full recipe

Forward

$$x \rightarrow W_1x + b_1 \rightarrow \text{ReLU} \rightarrow h \rightarrow W_2h + b_2 \rightarrow \text{softmax} \rightarrow \hat{y} \rightarrow L$$

Backward

$$\frac{\partial L}{\partial o} \rightarrow \frac{\partial L}{\partial h} \rightarrow \frac{\partial L}{\partial h_{\text{pre}}} \rightarrow \nabla W_1, \nabla b_1$$

Update

$\theta \leftarrow \theta - \eta \nabla \theta$, for every parameter tensor,
same rule as always

Note

This recipe — forward, then chain-rule backward one layer at a time, then update — is the template for training *any* feedforward network, not just this two-layer one

Tip

In lab: walk this exact example's full forward and backward pass **by hand**, including the dead-unit gradient